
Aurea Mediocritas

Relazione del progetto di Programmazione ad Oggetti

A.A. 2025/26 – Università di Bologna

Pietro Dapporto

`pietro.dapporto@studio.unibo.it`

Nikita Piraino

`nikita.piraino@studio.unibo.it`

Mario Conventi

`mario.conventi@studio.unibo.it`

Federico Locatelli

`federico.locatelli3@studio.unibo.it`

26 maggio 2026

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	3
2	Design	5
2.1	Architettura	5
2.2	Design dettagliato	6
2.2.1	Pietro Dapporto – Modello delle carte e contenuti	6
2.2.2	Mario Conventi – Algoritmo di selezione delle carte	8
2.2.3	Federico Locatelli – Parametri reattivi, architettura della View e ciclo di vita	8
2.2.4	Nikita Piraino – Game loop, tempo e condizioni di fine partita	15
3	Sviluppo	20
3.1	Testing automatizzato	20
3.2	Note di sviluppo	21
3.2.1	Pietro Dapporto	21
3.2.2	Mario Conventi	21
3.2.3	Federico Locatelli	22
3.2.4	Nikita Piraino	22
4	Commenti finali	23
4.1	Autovalutazione e lavori futuri	23
4.1.1	Pietro Dapporto	23
4.1.2	Mario Conventi	24
4.1.3	Federico Locatelli	24
4.1.4	Nikita Piraino	24
4.2	Difficoltà incontrate e commenti per i docenti	24
A	Guida utente	25
B	Esercitazioni di laboratorio	26

Capitolo 1

Analisi

In questo capitolo si effettua l'analisi dei requisiti e del dominio applicativo del progetto *Aurea Mediocritas*, senza alcun riferimento al design o alle tecnologie implementative.

1.1 Descrizione e requisiti

Aurea Mediocritas è un'applicazione che riproduce le vicende di un Rettore universitario impegnato a gestire un Ateneo bilanciando iniziative da intraprendere e situazioni critiche da risolvere. Il gioco si ispira al gioco di strategia a carte *Reigns*: il giocatore impersona il Magnifico per la durata di tre anni accademici (sei semestri) e deve arrivare al termine del mandato mantenendo l'equilibrio fra i parametri che descrivono lo stato dell'Università: **Finanze**, **Studenti**, **Docenti** e **Reputazione**.

Prima di iniziare, il giocatore sceglie il proprio nome, la facoltà di appartenenza e il livello di difficoltà della partita. A ogni turno riceve una carta che descrive un evento (un'iniziativa proposta, una protesta studentesca, un'opportunità di finanziamento, eccetera) e deve scegliere fra due alternative. Ciascuna scelta modifica i valori dei parametri di gioco. La partita si conclude positivamente se il Rettore arriva al termine del proprio mandato senza che alcun parametro raggiunga un valore critico; si conclude negativamente nel momento in cui un qualsiasi parametro raggiunge il livello minimo o massimo.

Requisiti funzionali

- Generazione di carte (eventi), ciascuna con un effetto associato alla scelta del giocatore.
- Costruzione di un mazzo che modella la successione di eventi della partita.
- Risposta del giocatore agli eventi tramite una scelta binaria (approvazione o rifiuto).
- Aggiornamento dei parametri di gioco a seguito della scelta effettuata, con intensità variabile in base alla difficoltà selezionata.
- Gestione delle condizioni di fine partita: vittoria al termine del mandato; sconfitta al raggiungimento dei valori critici di un parametro (0 o 100).
- Sistema di difficoltà selezionabile (Facile, Normale, Difficile) che influenza sia il peso dei danni applicati ai parametri sia l'algoritmo di selezione delle carte.
- Sistema di carte figlie (*follow-up*): alcune decisioni programmano la comparsa di una carta correlata dopo un numero definito di turni.
- Riepilogo inter-semestrale: al termine di ogni semestre viene mostrata una schermata con i valori correnti dei quattro parametri.

- Popup del consigliere: quando un parametro attraversa una soglia critica (inferiore al 10% o superiore al 90%), appare un avviso contestuale con un messaggio narrativo.
- Schermata di login: il giocatore inserisce il proprio nome, la facoltà e la difficoltà prima di iniziare la partita.
- Possibilità di riavviare la partita al termine senza chiudere l'applicazione.

Requisiti funzionali opzionali

- Visualizzazione proporzionale dell'impatto della carta corrente tramite indicatori visivi sulle icone dei parametri (pallini di antepresa durante il trascinamento).
- Modalità "apocalittica" con eventi più fantasiosi.
- Easter egg ambientati nel contesto Unibo.

Requisiti non funzionali

- L'applicazione deve essere distribuita come singolo file JAR eseguibile su qualunque piattaforma con Java 21.
- L'interfaccia grafica deve essere reattiva e gradevole, con animazioni fluide nelle transizioni di stato (ingresso carte, aggiornamento parametri, overlay di fine partita).
- Il software deve poter essere avviato senza configurazioni esterne, partendo dal solo JAR.
- Il codice deve rispettare le regole di qualità statica (Checkstyle, PMD, SpotBugs) verificate automaticamente a ogni build Gradle.

1.2 Modello del Dominio

Le entità principali del dominio sono il *Rettore* (il giocatore), che gestisce un insieme di *Parametri* dell'Ateneo. Ogni parametro ha un nome, un livello numerico compreso fra 0 e 100, e uno stato (attivo o esaurito). I quattro parametri sono: Finanze, Studenti, Docenti e Reputazione.

Lo svolgimento della partita è scandito da una sequenza di *Carte* prelevate da un *Mazzo*. Ogni carta descrive un evento, è associata a un *Personaggio* che la presenta, e propone al giocatore due *Decisioni* alternative (approvazione o rifiuto). A ciascuna decisione è associato un insieme di *Effetti*, ognuno dei quali specifica quanto e in quale direzione un parametro viene modificato.

Alcune decisioni attivano un meccanismo di *Conseguenza*: la scelta effettuata pianifica la comparsa di una carta figlia dopo un numero definito di turni. La relazione tra carta padre, tipo di scelta e carta figlia è descritta da un *FollowUp*.

Il *Tempo di gioco* è organizzato in turni raggruppati in semestri; un mandato completo è composto da sei semestri di sei turni ciascuno. Ad ogni istante, lo *Stato di gioco* descrive l'andamento della partita: in corso (*running*), vinta (*won*) o persa (*lost*).

Prima di iniziare, il giocatore fornisce la propria *Identità* (nome, facoltà e livello di difficoltà). La *Difficoltà* influenza il peso degli effetti applicati ai parametri e la probabilità con cui vengono selezionate le carte di supporto.

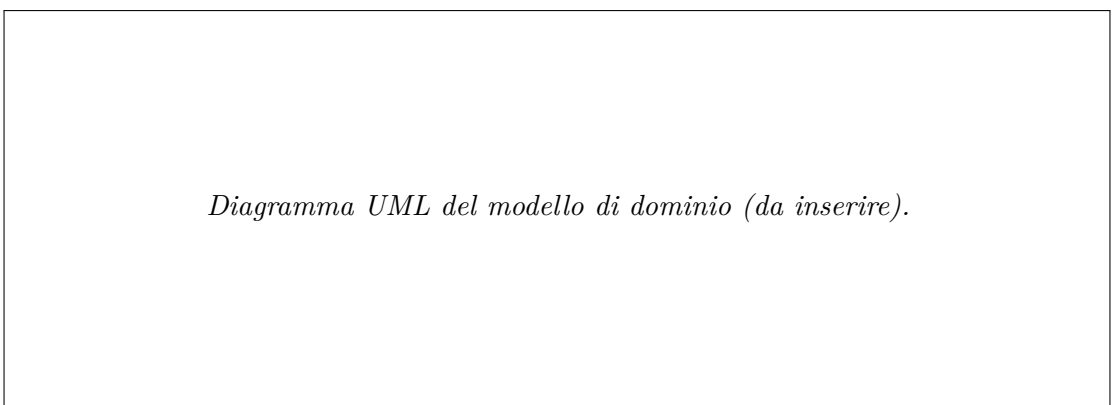


Diagramma UML del modello di dominio (da inserire).

Figura 1.1: Modello del dominio di Aurea Mediocritas: entità, attributi principali e relazioni.

Aspetti impegnativi L'aspetto più complesso da modellare è l'algoritmo di selezione delle carte: la successione non deve essere puramente casuale, ma deve evolvere in funzione dello stato corrente dei parametri, della difficoltà selezionata e delle conseguenze programmate dalle decisioni precedenti (follow-up). Un secondo aspetto impegnativo riguarda la reattività dell'interfaccia: la View deve aggiornarsi automaticamente a ogni variazione dei parametri, senza che il Model conosca l'esistenza della View. Infine, la gestione del ciclo di vita dell'applicazione (login, partita, fine partita, riavvio) richiede una coordinazione precisa fra le componenti senza introdurre dipendenze circolari.

Capitolo 2

Design

In questo capitolo si descrivono le strategie messe in campo per soddisfare i requisiti identificati in fase di analisi: prima la visione architetturale complessiva, poi gli aspetti di design di dettaglio più rilevanti svolti singolarmente da ciascun membro del gruppo.

2.1 Architettura

L'architettura dell'applicazione segue il pattern **Model-View-Controller (MVC)**, con una netta separazione delle responsabilità fra i tre livelli. Il confine fra le componenti è definito esclusivamente da interfacce: nessuno strato conosce l'implementazione concreta degli altri, solo il contratto esposto.

- Il **Model** (package `it.unibo.aurea.model`) racchiude la logica di gioco e lo stato dell'Ateneo. Il punto di ingresso è l'interfaccia `GameEngine`, che orchestra `GameConfig` (configurazione della partita), `GameClock` (progressione temporale), `Deck` (mazzo di carte) e i quattro `Parameter` (stato dell'Ateneo). Il Model non conosce né la View né il Controller.
- Il **Controller** (package `it.unibo.aurea.controller`) coordina il flusso di interazione tramite l'interfaccia `GameController`. Riceve le scelte del giocatore dalla View (`makeDecision()`), le inoltra al Model e aggiorna la View con il nuovo stato. Gestisce inoltre la connessione reattiva fra Model e View registrando gli observer sui parametri.
- La **View** (package `it.unibo.aurea.view`) implementa l'interfaccia `GameView` e presenta all'utente lo stato del gioco. Non accede mai direttamente al Model: riceve aggiornamenti tramite il pattern Observer (notifiche sui parametri) e tramite chiamate esplicite del Controller (`displayCard()`, `updateTime()`, `showVictory()`, ecc.).

Diagramma UML architetturale (MVC, da inserire).

Figura 2.1: Architettura complessiva del sistema: interfacce principali di Model, View e Controller.

La scelta di MVC garantisce che: (i) il Model sia completamente indipendente dalla libreria grafica utilizzata; (ii) la View possa essere sostituita in blocco — passando per esempio da JavaFX a un'interfaccia testuale — senza modificare Model o Controller; (iii) l'aggiunta di nuove funzionalità possa essere realizzata intervenendo soltanto sui componenti effettivamente coinvolti.

2.2 Design dettagliato

Questa sezione approfondisce le principali scelte progettuali adottate da ciascun membro del gruppo. Per ogni aspetto trattato vengono presentati: il problema da risolvere nel contesto specifico del progetto, la soluzione proposta con le motivazioni che l'hanno guidata, le alternative considerate e scartate, uno schema UML di supporto e l'identificazione del pattern di progettazione applicato, ove presente.

2.2.1 Pietro Dapporto – Modello delle carte e contenuti

Problema: modellazione delle carte di gioco

Nel gioco ogni carta rappresenta una proposta sottoposta al giocatore da un determinato personaggio. Si deve modellare una carta che non sia un semplice insieme di dati, ma che permetta di rappresentare in modo chiaro le due possibili decisioni del giocatore e le relative conseguenze sui parametri della partita.

Soluzione

La soluzione adottata consiste nel modellare la carta come un aggregato di oggetti più semplici, ciascuno con una responsabilità specifica. Una **Card** contiene un identificativo, una descrizione, il **Character** che la presenta e due oggetti **Decision**, rispettivamente associati al rifiuto e all'approvazione della proposta. Ogni **Decision** contiene il testo della risposta mostrata al giocatore e una collezione di **Effect**, che rappresentano le variazioni prodotte sui parametri del gioco.

In questo modo la carta non deve conoscere direttamente la logica di aggiornamento dei parametri, ma si limita a esporre le informazioni necessarie alla gestione della scelta.

Infine, il flag **usage** permette di distinguere le carte già utilizzate da quelle ancora disponibili, evitando che la stessa carta venga riproposta più volte durante la partita.

UML

Pattern utilizzati

In questa parte non è stato applicato un design pattern comportamentale in senso stretto. La scelta progettuale principale è basata sulla composizione: **Card** viene costruita aggregando oggetti più piccoli. L'uso di interfacce come **Card** ed **Effect** consente comunque di mantenere il modello estendibile, poiché eventuali implementazioni alternative potrebbero essere introdotte senza modificare il codice che dipende dalle astrazioni.

Problema: scalare la creazione delle carte

La creazione manuale di un numero elevato di carte direttamente nel codice avrebbe reso il sistema poco leggibile, difficile da modificare e soggetto a ripetizioni. Il problema principale era quindi permettere l'aggiunta e la modifica delle carte in modo semplice e mantenibile, senza dover intervenire ogni volta sulla logica Java del model.

Soluzione

In una prima fase sono state considerate diverse soluzioni basate sulla costruzione diretta degli oggetti nel codice, ad esempio tramite il costruttore di **CardImpl**, tramite **factory method** o

attraverso il pattern Builder. Queste soluzioni avrebbero però mantenuto la definizione delle carte troppo vicina al codice Java, rendendo meno immediata la lettura del contenuto di gioco e più costosa l'aggiunta di nuove carte.

Per questo motivo si è scelto di separare i dati dalla logica applicativa, spostando le informazioni delle carte in un file esterno in formato YAML. Il file contiene la descrizione dichiarativa delle carte, delle decisioni disponibili e degli effetti prodotti sui parametri del gioco. In questo modo il model non dipende dal modo in cui le carte sono scritte nel file, ma riceve oggetti già costruiti e coerenti con le prime astrazioni.

Il caricamento delle carte avviene all'interno della classe `Deck`, tramite metodi dedicati. Il file YAML viene deserializzato in una struttura intermedia rappresentata da `CardsFile`, che contiene una collezione di `CardDTO`: ciascuna contiene le decisioni `DecisionDTO` che a loro volta contengono i relativi `EffectDTO`. Durante questa fase, i dati letti dal file vengono convertiti dagli oggetti DTO agli oggetti effettivi del dominio, come `Card`, `Decision` ed `Effect`.

Questa scelta migliora la mantenibilità perché nuove carte possono essere aggiunte o modificate intervenendo principalmente sul file di configurazione. La soluzione introduce però anche un livello di astrazione aggiuntivo: potenzialmente sarebbe possibile convertire le informazioni del file YAML in modelli di carta differenti, ma nel progetto attuale tutte le carte vengono ricondotte allo stesso modello interno.

UML

Pattern utilizzati

In questa parte non è stato applicato un design pattern classico. Tuttavia per gestire il passaggio tra il file YAML e gli oggetti del model è stato utilizzato il concetto di DTO (*Data Transfer Object*). Un DTO è un oggetto usato per trasferire dati tra due parti del sistema senza contenere logica di dominio significativa. In questo ambito agiscono quindi come classi ponte tra la rappresentazione esterna delle carte, definita nel file YAML, e la rappresentazione interna usata dal model.

Problema: gestione delle carte figlie

Il gioco deve permettere che alcune decisioni producano conseguenze non immediate, facendo apparire nei turni successivi una carta collegata alla scelta effettuata. Inoltre la carta figlia deve apparire solo dopo una decisione su un'altra carta e non casualmente come una normale carta del mazzo principale.

Soluzione

Per implementare questo comportamento è stato introdotto il concetto di `FollowUp`, anch'esso definito tramite file YAML. Un `FollowUp` rappresenta una relazione logica tra una carta padre e una carta figlia, identificandole tramite i rispettivi id univoci. Oltre agli identificativi, il `FollowUp` specifica il trigger che attiva la relazione, cioè la scelta necessaria tra approvazione e rifiuto, e il numero di turni di ritardo dopo i quali la carta figlia potrà essere proposta.

Le carte figlie sono mantenute in un file YAML separato rispetto al mazzo principale. Durante il caricamento, il mazzo principale e le carte figlie vengono quindi convertiti in strutture dati distinte: il primo rappresenta le carte pescabili normalmente, mentre la seconda struttura contiene le carte disponibili solo come conseguenza di un `FollowUp`.

Quando il giocatore prende una decisione su una carta, il sistema verifica se esiste un `FollowUp` compatibile con l'id della carta e con la scelta effettuata. In caso positivo, la carta figlia viene programmata per comparire dopo il numero di turni indicato.

UML

Pattern utilizzati

In questa parte non è stato applicato un design pattern classico in senso stretto. La soluzione si basa principalmente sulla separazione delle responsabilità e sulla rappresentazione esplicita delle relazioni tra carte tramite configurazione esterna.

2.2.2 Mario Conventi – Algoritmo di selezione delle carte

[Sezione da redigere: progettazione del package `model.selection`, `CardSelector` e implementazioni (`WeightedRandomSelector`, `ProgressiveSelector`), gestione di cooldown e storico, identificazione del pattern `Strategy`.]

2.2.3 Federico Locatelli – Parametri reattivi, architettura della View e ciclo di vita

La mia responsabilità copre tre livelli del sistema: la gestione reattiva dei parametri nel Model, l'intero layer View con le sue animazioni e componenti interattivi, e l'estensione del Controller con nuovi metodi di anteprima e accesso allo stato. Di seguito una panoramica dei componenti realizzati.

Compito	Interfaccia	Implementazione
Parametro (sola lettura, View layer)	<code>ParameterView</code>	<code>ParameterImpl</code>
Parametro (mutabile, Model layer)	<code>Parameter</code>	<code>ParameterImpl</code>
Observer dei parametri	<code>ParameterObserver</code>	lambda / method reference
Tipo parametro con comportamento	—	<code>ParameterType</code> (enum)
Identità giocatore	—	<code>PlayerInfo</code> (record)
Contratto del Controller	<code>GameController</code>	aggiunti a <code>GameControllerImpl</code>
View principale	<code>GameView</code>	<code>GameViewJavaFXImpl</code>
Carta interattiva swipeable	—	<code>CardPanel</code>
Icona parametro animata	—	<code>ParameterIconView</code>
Schermata di login	—	<code>LoginScene</code>
Popup del consigliere	—	<code>CounsellorPopup</code>
Overlay di fine partita (in collaborazione)	—	<code>EndgameOverlay</code>
Stile visivo dell'applicazione	—	<code>styles.css</code>

Tabella 2.1: Componenti realizzati da Federico Locatelli.

Pattern Observer: parametri reattivi

Problema: La View deve aggiornarsi ogni volta che il valore di un parametro cambia, senza che il Model conosca l'esistenza della View. Un accoppiamento diretto fra le due componenti violerebbe il principio di separazione delle responsabilità dell'architettura MVC e renderebbe impossibile sostituire la View senza modificare il Model.

Soluzione: È stato adottato il pattern **Observer**. L'interfaccia `ParameterObserver` è dichiarata `@FunctionalInterface`: definisce un unico metodo `onParameterChanged(ParameterType, int)` che viene invocato ogni volta che il livello del parametro cambia. `ParameterImpl` funge da *observable*: mantiene una lista di osservatori registrati e li notifica al termine di ogni

`modify()`, usando una copia difensiva della lista (`List.copyOf`) per prevenire eccezioni di modifica concorrente.

La registrazione dell'osservatore avviene nel costruttore di `GameControllerImpl`, dove la View viene collegata al Model tramite una *method reference*:

```
parametersMap.values()
    .forEach(p -> p.addObserver(this.view::updateSingleParameter));
```

In questo modo il Model non conosce la View: conosce solo l'interfaccia `ParameterObserver`, che può essere implementata da qualsiasi oggetto o, come in questo caso, da una lambda o method reference.

Alternative considerate:

- *Polling*: la View interroga periodicamente il Model per verificare se i valori sono cambiati. Scartato perché introduce latenza, spreco di cicli CPU e accoppiamento temporale.
- *Chiamata diretta View → Model*: il Controller aggiorna la View dopo ogni decisione. Parzialmente adottato per l'inizializzazione, ma insufficiente per aggiornamenti intermedi (es. effetti concatenati su più parametri).

UML Observer: ParameterImpl (observable), ParameterObserver (interfaccia), GameViewJavaFXImpl (observer via method reference). Da inserire.

Figura 2.2: Pattern Observer applicato ai parametri di gioco.

Pattern identificato – Observer: `ParameterImpl` è l'*observable*; `ParameterObserver` è l'interfaccia dell'*observer*; `GameViewJavaFXImpl` (`updateSingleParameter`) è l'*observer concreto*, registrato tramite method reference dal Controller.

Interface Segregation: separazione tra lettura e scrittura dei parametri

Problema: Il Model espone i parametri sia alla View (che deve solo leggerli per aggiornare l'interfaccia) sia al Controller (che deve modificarli applicando gli effetti delle carte). Esporre un'unica interfaccia con tutti i metodi, incluso `modify()`, avrebbe consentito alla View di alterare direttamente lo stato del gioco, violando i confini dell'architettura MVC.

Soluzione: È stato applicato l'**Interface Segregation Principle**: i metodi del parametro sono stati suddivisi in due interfacce distinte con responsabilità diverse.

`ParameterView` espone esclusivamente i metodi di consultazione: `getLevel()`, `isAlive()`, `getName()`, `getDeathReason()` e `addObserver()`. Questo è il tipo che la View e il Controller utilizzano per leggere lo stato dei parametri senza rischiare modifiche accidentali.

`Parameter` estende `ParameterView` aggiungendo il solo metodo `modify(int delta)`, che altera il livello del parametro. Questo tipo è utilizzato esclusivamente all'interno del `Model`.

La distinzione è resa esplicita nel `Controller`, dove la mappa dei parametri è tipizzata come `Map<ParameterType, ParameterView>`: il `Controller` legge i livelli e registra gli `observer`, ma non chiama mai `modify()` direttamente — delega questa responsabilità al `Model` tramite `model.applyEffects()`.

Alternative considerate:

- *Interfaccia unica*: esporre tutti i metodi in una sola interfaccia. Più semplice, ma permette alla `View` di chiamare `modify()` per errore, introducendo bug difficili da diagnosticare.
- *Copia difensiva al getter*: restituire una copia dei parametri invece di riferimenti diretti. Scartato perché rompe il collegamento con il pattern `Observer` — gli `observer` registrati sulla copia non riceverebbero le notifiche del parametro originale.

UML ISP: `ParameterView` (sola lettura), `Parameter` estende `ParameterView` aggiungendo `modify()`, `ParameterImpl` implementa entrambe. Da inserire.

Figura 2.3: Interface Segregation applicata ai parametri di gioco.

Principio identificato – Interface Segregation (ISP): `ParameterView` è l'interfaccia *ristretta* usata da `View` e `Controller`; `Parameter` è l'interfaccia *completa* usata dal `Model`; `ParameterImpl` implementa entrambe.

Type-Safe Enum con comportamento: `ParameterType`

Problema: I quattro parametri del gioco devono essere identificati in modo univoco e presentati all'utente con un nome leggibile. Una soluzione ingenua avrebbe usato stringhe letterali ("`finances`", "`students`", ecc.) o avrebbe applicato trasformazioni come `name().toLowerCase()` direttamente nel codice della `View`, introducendo dipendenze fragili dal nome interno dell'enum e duplicazione della logica di presentazione.

Soluzione: È stato adottato il pattern **Type-Safe Enum con comportamento**: l'enum `ParameterType` non è un semplice elenco di etichette, ma un oggetto che conosce come presentarsi. Ogni costante porta con sé un campo `displayName` inizializzato nel costruttore e accessibile tramite il metodo `getDisplayname()`:

```
public enum ParameterType {  
    FINANCES("Finances"),  
    STUDENTS("Students"),  
    PROFESSORS("Professors"),  
    REPUTATION("Reputation");  
}
```

```

private final String displayName;

ParameterType(final String displayName) {
    this.displayName = displayName;
}

public String getDisplayName() {
    return displayName;
}
}

```

In questo modo la View chiama `type.getDisplayName()` invece di manipolare `type.name()`: il codice di presentazione è disaccoppiato dall'identificatore interno dell'enum. Se in futuro si volesse rinominare la costante `FINANCES` in `MONEY`, il nome mostrato all'utente rimarrebbe invariato senza toccare la View.

Alternative considerate:

- *Metodo statico di mappatura*: una `Map` o uno `switch` separato che traduce l'enum in stringa. Scartato perché distribuisce la logica di presentazione in un punto lontano dalla definizione del tipo.
- *Override di `toString()`*: restituire il display name da `toString()`. Scartato perché `toString()` è un metodo generico con semantica ambigua; un metodo dedicato `getDisplayName()` rende l'intenzione esplicita.

*UML ParameterType: enum con campo displayName e metodo getDisplayName().
Da inserire.*

Figura 2.4: Type-Safe Enum con comportamento applicato a ParameterType.

Pattern identificato – Type-Safe Enum con comportamento: `ParameterType` non è solo un'etichetta ma un oggetto con logica di presentazione incapsulata. Ogni costante è *self-describing*: sa come mostrarsi alla View senza richiedere logica esterna.

Pattern Strategy: comportamenti iniettati in CardPanel

Problema: `CardPanel` gestisce l'interazione fisica con la carta (trascinamento, rotazione, volo), ma deve anche: notificare il Controller quando il giocatore prende una decisione, richiedere un'anteprima dei parametri colpiti durante il trascinamento, e segnalare quando l'anteprima termina. Implementare questi comportamenti direttamente in `CardPanel` avrebbe creato una dipendenza diretta verso il Controller, accoppiando il componente visuale alla logica di gioco e rendendo impossibile testarlo in isolamento.

Soluzione: È stato adottato il pattern **Strategy** tramite interfacce funzionali Java: i tre comportamenti variabili sono rappresentati da campi di tipo `BiConsumer`, `Function` e `Runnable`, iniettati dall'esterno tramite metodi setter:

```
private BiConsumer<Card, Boolean> onDecision;
private Function<Boolean, Set<ParameterType>> previewProvider;
private Runnable onPreviewEnd;
```

`CardPanel` non conosce il `Controller`: si limita a invocare le strategie ricevute nei momenti opportuni. La configurazione avviene in `GameViewJavaFXImpl`, che inietta le implementazioni concrete tramite method reference verso il `Controller`:

```
cardPanel.setOnDecision((card, approved) ->
    controller.makeDecision(approved));
cardPanel.setPreviewProvider(
    controller::previewDecisionDeltas);
cardPanel.setOnPreviewEnd(this::resetHighlights);
```

Questo approccio permette di sostituire i comportamenti senza modificare `CardPanel` e di testarlo con implementazioni fittizie (stub) senza coinvolgere il `Controller` reale.

Alternative considerate:

- *Dipendenza diretta da GameController*: passare il `Controller` come parametro al costruttore di `CardPanel`. Scartato perché lega il componente visuale a una specifica interfaccia del `Controller`, riducendo la riusabilità.
- *Listener ad hoc con interfacce dedicate*: definire interfacce `DecisionListener`, `PreviewListener` ecc. Più verboso senza vantaggi concreti rispetto all'uso diretto delle interfacce funzionali di Java.

UML Strategy: CardPanel con i tre campi funzionali (BiConsumer, Function, Runnable) iniettati da GameViewJavaFXImpl. Da inserire.

Figura 2.5: Pattern Strategy applicato ai comportamenti di `CardPanel`.

Pattern identificato – Strategy: `CardPanel` è il *context*; `BiConsumer`, `Function` e `Runnable` sono le *strategie*; le lambda e method reference iniettate da `GameViewJavaFXImpl` sono le *strategie concrete*.

Inversion of Control: gestione del ciclo di vita

Problema: Al termine di una partita il giocatore può scegliere di ricominciare da capo. La `View`, che gestisce la schermata di fine partita, conosce il momento in cui il giocatore preme “Reign Again”, ma non dovrebbe sapere *come* riavviare il gioco: farlo significherebbe che la `View` conosce la struttura dell'intera applicazione (Model, Controller, login), violando la separazione delle responsabilità MVC.

Soluzione: È stato applicato il principio di **Inversion of Control** tramite callback: la View riceve un oggetto `Runnable onRestart` nel costruttore, senza sapere cosa fa. Quando il giocatore preme il tasto di riavvio, la View chiude gli stage aperti e invoca il callback, delegando completamente la responsabilità a chi lo ha fornito.

Il proprietario del ciclo di vita è `Main`, che definisce il callback come una method reference verso il proprio metodo `openLogin()`:

```
final Runnable onRestart = () ->
    Platform.runLater(this::openLogin);
final GameView view =
    new GameViewJavaFXImpl(onRestart);
```

Quando il callback viene eseguito, `Main` chiude il gioco corrente e apre una nuova `LoginScene`, ricreando l'intero stack MVC con i nuovi dati del giocatore. La View non conosce nulla di questo flusso: esegue un `Runnable` e basta.

Alternative considerate:

- *Riferimento diretto a Main:* passare l'istanza di `Main` alla View. Scartato perché crea una dipendenza ciclica e concettualmente errata: la View non dovrebbe conoscere il punto di ingresso dell'applicazione.
- *Riavvio tramite Platform.exit() e riapertura:* chiudere la JVM e riaprire l'applicazione. Scartato perché non è trasparente all'utente e non è un riavvio autentico all'interno della stessa sessione.

*UML IoC: Main fornisce Runnable onRestart a GameViewJavaFXImpl;
EndgameOverlay lo invoca tramite la View senza conoscere Main. Da inserire.*

Figura 2.6: Inversion of Control per la gestione del riavvio.

Principio identificato – Inversion of Control: `Main` è il *proprietario del ciclo di vita*; `GameViewJavaFXImpl` è il *consumatore del callback*; il `Runnable` è il *contratto minimo* che disaccoppia i due senza che la View conosca `Main`.

Value Object: `PlayerInfo` come record immutabile

Problema: I dati inseriti dal giocatore nella schermata di login — nome del Rettore, facoltà e difficoltà — devono essere raccolti, validati e trasportati attraverso il sistema fino al Controller, che li utilizza per personalizzare l'esperienza di gioco. Una soluzione basata su un oggetto mutabile avrebbe esposto il rischio di modifiche accidentali durante il transito, e la validazione sarebbe potuta essere dimenticata in qualunque punto del codice.

Soluzione: È stato adottato il pattern **Value Object** tramite il costrutto `record` di Java 21. `PlayerInfo` è dichiarato come record con tre componenti: `rectorName`, `faculty` e `difficulty`. Il costruttore compatto applica la validazione *fail-fast* su tutti i campi:

```
public record PlayerInfo(
    String rectorName,
    String faculty,
    Difficulty difficulty) {

    public PlayerInfo {
        Objects.requireNonNull(rectorName,
            "rectorName must not be null");
        Objects.requireNonNull(faculty,
            "faculty must not be null");
        Objects.requireNonNull(difficulty,
            "difficulty must not be null");
    }
}
```

Il compilatore genera automaticamente `equals()`, `hashCode()` e `toString()` coerenti con i valori dei campi, e i campi sono implicitamente `final`: una volta creato, un `PlayerInfo` non può essere modificato. Questo garantisce che i dati del giocatore siano consistenti in ogni punto del sistema che li utilizza.

Alternative considerate:

- *Classe mutabile con getter e setter*: più flessibile ma priva delle garanzie di immutabilità. La validazione dovrebbe essere replicata in ogni setter, aumentando il rischio di inconsistenza.
- *Parametri separati*: passare nome, facoltà e difficoltà come argomenti separati al costruttore del Controller. Scartato perché aumenta l'aridità dei metodi e non raggruppa semanticamente i dati correlati.

UML PlayerInfo: record con tre campi immutabili e costruttore compatto con validazione. Da inserire.

Figura 2.7: Value Object `PlayerInfo` come Java record.

Pattern identificato – Value Object: `PlayerInfo` è immutabile, si confronta per valore e trasporta dati validati. Il costrutto `record` di Java 21 fornisce queste garanzie con codice minimo e senza boilerplate.

Ulteriori scelte di design

Oltre ai pattern principali descritti nelle sezioni precedenti, alcune scelte progettuali minori meritano una menzione per il loro impatto sulla qualità e manutenibilità del codice.

Factory Method in GameViewJavaFXImpl: La costruzione dei componenti visivi della View è delegata a metodi privati dedicati: `buildTopSection()`, `buildCenterSection()`, `buildBottomSection()`, `buildCloseButton()`. Ogni metodo è responsabile di costruire e restituire un nodo JavaFX autocontenuto. Questo approccio riduce la complessità del metodo principale `buildAndShowStage()` e permette di modificare un componente senza impattare gli altri.

Utility Class: CounsellorPopup: `CounsellorPopup` è progettata come *utility class*: costruttore privato, nessun campo di istanza, unico metodo pubblico statico `show(String message)`. La scelta è motivata dalla natura *stateless* del popup: ogni invocazione costruisce un nuovo stage modale, lo mostra e lo chiude senza mantenere stato tra una chiamata e l'altra.

Fail-Fast e Defensive Copy in ParameterImpl: Il costruttore di `ParameterImpl` e il metodo `addObserver()` applicano `Objects.requireNonNull` per rigettare immediatamente input nulli, evitando errori latenti difficili da diagnosticare. Il metodo `notifyObservers()` itera su una copia difensiva della lista degli osservatori (`List.copyOf`), rendendo il sistema robusto rispetto a modifiche concorrenti della lista durante la notifica.

Gestione del threading in JavaFX: Tutti i metodi della View che modificano nodi della scena grafica (`displayCard()`, `updateSingleParameter()`, `showVictory()`, ecc.) delegano l'esecuzione al thread JavaFX tramite `Platform.runLater()`. Questo garantisce che gli aggiornamenti dell'interfaccia avvengano sempre sul thread corretto, prevenendo eccezioni di accesso concorrente tipiche delle applicazioni JavaFX multi-thread.

2.2.4 Nikita Piraino – Game loop, tempo e condizioni di fine partita

La parte di progetto di mia responsabilità copre la logica di stato della partita e il ciclo temporale del gioco:

Compito	Nome interfaccia	Nome implementazione
Configurazione partita	GameConfig	GameConfigImpl
Gestore turni	GameClock	GameClockImpl
Orchestratore logica gioco	GameEngine	GameEngineImpl
Pagina iniziale	VERIFICARE	VERIFICARE
Report sullo stato dei parametri	Report	ReportImpl
Pagina vittoria o sconfitta	VERIFICARE	VERIFICARE

Pattern Static Factory Method in GameConfigImpl

Problema: Sono previste più versioni di configurazione del gioco: una standard (6 carte per semestre, 6 semestri), una ridotta (2 carte, 2 semestri) pensata per facilitare i test intermedi e di fine partita. C'è anche il desiderio di implementare future versioni alternative di partite come ad esempio la modalità “apocalisse”. Occorre quindi controllare l'istanziamento della configurazione, vincolando gli utilizzatori a versioni standardizzate di partita, facilmente scegliibili tramite i metodi pubblici e statici della classe.

Soluzione: La classe `GameConfigImpl` adotta il pattern Static Factory Method articolato in tre elementi sinergici:

- Costruttore privato. Il costruttore `GameConfigImpl(int cardsPerSemester, int semestersPerGame)` è dichiarato `private`: impedisce l'istanziamento diretta da parte dei client e concentra il controllo della validità dei parametri in un unico punto.
- Costanti di configurazione private. I valori numerici delle due modalità sono incapsulati in quattro costanti statiche private di tipo `int`:

```
private static final int STANDARD_CARDS_PER_SEMESTER = 6;
private static final int STANDARD_SEMESTERS_PER_GAME = 6;
private static final int MINIMAL_NUMBER_SEMESTER = 2;
private static final int MINIMAL_NUMBER_CARDS = 2;
```

L'uso di `static final` garantisce l'unicità del valore in memoria e ne impedisce la modifica a runtime; l'ordinamento rispetta la regola Checkstyle Declaration Order (costanti statiche prima dei campi di istanza).

- Factory method pubblici e statici. `createStandard()` produce la configurazione normale, `createShort()` la ridotta per testing. Il tipo di ritorno dichiarato è l'interfaccia `GameConfig` (non l'implementazione concreta): i client dipendono dall'astrazione, conformemente al Dependency Inversion Principle. Ciò rende possibile introdurre nuove configurazioni (per esempio una "Modalità Apocalisse") senza modificare i client. La classe è dichiarata `final`: la sottoclasse è impedita e l'invariante di immutabilità di ogni istanza creata viene rafforzato.

Pattern identificato Static Factory Method (creazionale): i metodi `createStandard()` e `createShort()` sono le factory; `GameConfig` è il tipo prodotto; `GameConfigImpl` è l'implementazione concreta resa inaccessibile direttamente dal client.

Orchestrazione del modello in `GameEngineImpl`

Problema: La logica di partita richiede di coordinare più oggetti collaboranti: configurazione, orologio, mazzo, parametri. Occorre un punto di ingresso unico per il controller, garantendo immutabilità strutturale del modello dopo la costruzione e fail-fast sulle dipendenze nulle.

Soluzione: `GameEngineImpl` implementa `GameEngine` ed è la classe centrale del sottosistema model. Coordina tre dipendenze: la configurazione (`GameConfig`), l'orologio (`GameClock`) e il mazzo (`Deck`). Tutti i campi sono `final`; in particolare `parameters` è inizializzato con `List.of(...)`, che restituisce una lista non modificabile: nessun client può aggiungere o rimuovere parametri dopo la costruzione dell'istanza.

Il costruttore applica la Dependency Injection ricevendo `GameConfig` e `Deck` come argomenti, e crea internamente il `GameClockImpl`:

```
public GameEngineImpl(final GameConfig config, final Deck deck) {
    this.config = config;
    this.gameClock = new GameClockImpl(config);
    this.deck = Objects.requireNonNull(deck, "Deck cannot be null");
}
```

La chiamata a `Objects.requireNonNull` applica il principio fail-fast: se il client passa `null`, una `NullPointerException` viene sollevata immediatamente nel costruttore con un messaggio esplicativo, evitando errori latenti che si manifesterebbero solo durante l'esecuzione.

Il metodo `getCurrentCard()` è stato refattorizzato per utilizzare lo Stream API al posto di un accesso diretto per indice, individuando la prima carta non ancora utilizzata nel mazzo:

```
return deck.getAllCards().stream()
    .filter(card -> !card.isUsed())
    .findFirst()
    .orElseGet(() -> deck.getAllCards().get(0));
```

Il terminale `orElseGet` esegue il fallback solo se lo stream è vuoto (tutte le carte sono state utilizzate), restituendo la prima carta come comportamento di default.

Alcune alternative sono:

- Accesso per indice: non scelto perché richiede un controllo esplicito sulla lista vuota e rende esplicita nel codice l'intenzione del metodo.

Eventuali pattern: nessuno in senso stretto oltre alla Dependency Injection.

State Machine in `getGameState()`

Problema: Lo stato di una partita può essere in corso, vinta o persa; queste tre alternative sono mutuamente esclusive e dipendono dallo stato corrente dei parametri e dell'orologio. La consultazione dello stato è frequente (a ogni interazione del controller con il modello), per cui un campo di stato esplicito introdurrebbe rischi di disallineamento fra i dati interni e la valutazione restituita.

Soluzione: Il metodo `getGameState()` nella classe `GameEngineImpl` avvia un processo di identificazione dello stato, che poi restituirà uno dei valori dell'enumerazione `GameState`: `RUNNING`, `WON`, `LOST`.

```
public GameState getGameState() {
    if (!areAllParametersAlive()) {
        return GameState.LOST;
    }
    if (gameClock.isTimeFinished()) {
        return GameState.WON;
    }
    return GameState.RUNNING;
}
```

L'ordine dei controlli è semanticamente obbligatorio: il controllo `LOST` (morte di un parametro) precede `WON` (tempo esaurito) perché nell'ultimo turno di gioco entrambe le condizioni possono essere simultaneamente vere; in tal caso la morte di un parametro deve prevalere sulla vittoria per tempo.

Lo stato viene calcolato ad ogni chiamata: l'approccio è stateless, elimina il rischio di stato inconsistente tra le transizioni e non richiede aggiornamenti espliciti dopo ogni operazione.

Alcune alternative sono:

- Restituire una stringa al posto degli elementi dell'enum: non idoneo con gli standard di programmazione moderni.
- Nelle condizioni non richiamare delle funzioni ma mettere il loro contenuto direttamente nel metodo `getGameState()`: supporta meno per necessità future i principi DRY e SRP.

Stream API e `areAllParametersAlive()`

Problema: La condizione di sconfitta richiede di verificare che tutti i parametri siano vivi. La verifica deve essere efficiente (idealmente interrompendo l'analisi al primo parametro non valido) e leggibile.

Soluzione: Il metodo privato `areAllParametersAlive()` nella classe `GameEngineImpl` incapsula la verifica:

```
private boolean areAllParametersAlive() {
    return parameters.stream().allMatch(Parameter::isAlive);
}
```

Il terminale `allMatch` applica il predicato `isAlive()` a ogni elemento tramite method reference. Restituisce `true` solo se il predicato è soddisfatto da tutti gli elementi, e interrompe l'elaborazione al primo elemento che lo viola (Fail Fast Principle).

Alcune alternative sono:

- Utilizzare un ciclo `for` imperativo: avrebbe potuto adottare il fail-fast, ma sarebbe stato più verboso.
- Usare lo stream senza `allMatch`: non avrebbe adottato la logica di corto circuito del fail-fast.

Progressione temporale in `GameClockImpl`

Problema: La progressione del tempo deve trasportare tre informazioni: turno corrente nel semestre, semestre corrente, terminazione della partita. La logica di avanzamento deve garantire che il flusso turno -> semestre successivo -> fine partita avvenga deterministicamente e che non sia possibile incrementare l'orologio dopo la fine del mandato.

Soluzione: `GameClockImpl` gestisce il ciclo temporale tramite le variabili di stato `currentTurn`, `currentSemester` e `timeFinished`. Il metodo `nextTurn()` implementa la logica di avanzamento con una catena `if/else if/else` che garantisce che esattamente uno dei tre rami venga eseguito per ogni chiamata:

```
public void nextTurn() {
    if (timeFinished) {
        throw new IllegalStateException("Game already finished");
    }
    if (hasNextTurnInSemester()) {
        currentTurn++;
    } else if (hasNextSemester()) {
        currentSemester++;
        currentTurn = 0;
    } else {
        timeFinished = true;
    }
}
```

La condizione iniziale su `timeFinished` implementa il fail-fast principle per chiamate illegali su un orologio già terminato. I due metodi privati `hasNextTurnInSemester()` e `hasNextSemester()` verificano in modo isolato le condizioni di avanzamento, mantenendo bassa la complessità ciclomatica di `nextTurn()`.

Alcune alternative sono:

- Scrivere il codice delle funzioni dentro le condizioni: riduce la flessibilità del codice per futuri aggiornamenti e non supporta pienamente il principio DRY.

Design di `ReportImpl` come componente `JavaFX` riutilizzabile

Problema: Alla fine di ogni semestre e alla fine della partita si vuole presentare un report al giocatore nel quale sono presenti i valori di ogni parametro e delle frasi di storytelling.

Soluzione: `ReportImpl` è un componente della View e implementa l'interfaccia `Report`. È progettato per essere riutilizzato sia come schermata di riepilogo inter-semestrale sia come schermata di fine partita, senza creare istanze multiple.

Alcune alternative sono:

- Pattern Strategy: Definire un'interfaccia comune per il report e realizzare implementazioni separate per le diverse tipologie di riepilogo. Questa alternativa non è stata scelta poiché

le differenze grafiche e logiche tra i report sono attualmente minime, e l'introduzione del pattern avrebbe comportato solo una maggiore complessità del codice.

- Creare elementi al livello sopra: invece di cancellare con `.children().clear()` si aggiunge un nuovo report sopra. Permetterebbe di creare delle animazioni in cui si mostra la modifica dei parametri, però richiederebbe molto più utilizzo di memoria.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il testing automatizzato è stato realizzato con **JUnit 5 (Jupiter)**, con test posti in `src/test/java` e package paralleli a quelli dei sorgenti. I test non richiedono alcuna interazione utente e sono eseguiti automaticamente dal task `gradle build`.

I componenti sottoposti a test automatizzato comprendono il sottosistema model in tutte le sue parti: configurazione, orologio, motore di gioco, mazzo di carte, regole di applicazione degli effetti e algoritmo di selezione. Per ciascun membro del gruppo, le strategie adottate sono descritte in maggior dettaglio nella sezione di note di sviluppo individuale.

Nikita Piraino – strategia di testing per transizioni di stato La strategia adottata è il *State Transition Testing*: ogni test verifica che il sistema si trovi nello stato corretto prima e dopo un'operazione che ne altera il comportamento. L'annotazione `@BeforeEach` garantisce l'isolamento fra test: ogni metodo riceve un'istanza fresca del sistema sotto test, eliminando dipendenze di ordine.

Le classi di test coprono tre classi di implementazione: `GameEngineTest` (transizioni di stato), `GameClockTest` (progressione temporale), `GameConfigTest` (validità della configurazione).

GameEngineTest `setUp()` inizializza il motore con la configurazione standard e un mazzo vuoto:

```
this.engine = new GameEngineImpl(GameConfigImpl.createStandard(), new Deck());
```

- `testGameInitialization()`: verifica che lo stato iniziale sia `RUNNING` prima e dopo `engine.start()`, documentando che l'inizializzazione non altera lo stato logico del gioco.
- `testGameLostOnParameterDeath()`: simula la morte del parametro `FINANCES` azzerandone il livello; verifica che `areAllParametersAlive()` rilevi il parametro morto e che `getGameState()` risponda con `LOST`.
- `testGameWonOnTimeFinished()`: verifica la transizione `RUNNING -> WON` esaurendo il tempo tramite un ciclo `while (!isTimeFinished()) nextTurn();`, controllando anche la collaborazione fra `GameEngineImpl` e `GameClockImpl`.

GameClockTest `configuration()` inizializza orologio e configurazione con `createStandard()`.

- `testSingleTurnProgression()`: verifica che una singola `nextTurn()` incrementi `currentTurn` di 1 senza alterare `currentSemester`. La cattura dei valori iniziali rende l'asserzione indipendente dalla configurazione specifica.

- `testSemesterFlow()`: verifica che dopo `cardsPerSemester` chiamate a `nextTurn()` il semestre avanzi di 1 e il contatore dei turni venga azzerato a 0.
- `testEndofTime()`: esegue il ciclo completo dei turni e verifica che `isTimeFinished()` diventi `true` al termine di tutti i semestri.

GameConfigTest `testStandardConfigurationValidity()` verifica che `getCardsPerSemester()` e `getSemestersPerGame()` restituiscano valori strettamente positivi per la configurazione standard. Il contratto coperto è minimo per design: verificare i valori esatti (6 e 6) renderebbe il test fragile rispetto a future variazioni della configurazione standard.

Pietro Dapporto – testing caricamento elementi YAML I test riguardano principalmente il sistema di carte e il caricamento dei dati da file YAML. Vista la brevità ho raggruppato in un'unica classe più test.

Sono stati sottoposti a test i seguenti aspetti:

- `shouldLoadCards()`: verifica che le informazioni scritte nei file YAML coincidano con ciò che è contenuto dentro l'implementazione delle carte create.
- `shouldLoadFollowUp()`: esegue la stessa verifica ma sui followup.
- `shouldHaveUniqueIdCards()`: verifica che le carte base abbiano tutte un identificatore (`String id`) diverso.
- `shouldHaveUniqueIdChildCards()`: esegue la stessa verifica ma sulle carte figlie.
- `shouldNotBeSharedIds()`: verifica che non ci siano identificatori comuni tra le carte base e quelle figlie.
- `shouldCharacterImageExist()`: verifica che ogni personaggio all'interno delle carte abbia il riferimento ad un'immagine da mostrare durante il gioco.

Mario Conventi, Federico Locatelli *[Sezioni di testing individuali da redigere.]*

3.2 Note di sviluppo

3.2.1 Pietro Dapporto

- **Libreria esterna Jackson YAML**
<https://github.com/Federicolocaa/00P25-Aurea/blob/dd18617aeefb0d0e4c01236c24cc8c7b8c0a8/src/main/java/it/unibo/aurea/model/Deck.java#L31-L82>
- **Record Java come Data Transfer Object** - usati per `CardDTO`, `EffectDTO`, `DecisionDTO`, `FollowUpDTO`, `FollowUpsFile`, `CardsFile` di seguito solo un esempio.
<https://github.com/Federicolocaa/00P25-Aurea/blob/dd18617aeefb0d0e4c01236c24cc8c7b8c0a8/src/main/java/it/unibo/aurea/model/dto/CardDTO.java#L5-L21>
- **Stream API e method reference** - usati anche nei test automatizzati (`CardLoaderTest`)
<https://github.com/Federicolocaa/00P25-Aurea/blob/dd18617aeefb0d0e4c01236c24cc8c7b8c0a8/src/main/java/it/unibo/aurea/model/Deck.java#L58-L78>

3.2.2 Mario Conventi

[Sezione da redigere: feature avanzate del linguaggio Java utilizzate, ciascuna con permalink GitHub al punto del codice.]

3.2.3 Federico Locatelli

[Sezione da redigere: feature avanzate del linguaggio Java utilizzate, ciascuna con permalink GitHub al punto del codice.]

3.2.4 Nikita Piraino

Di seguito le feature avanzate del linguaggio e dell’ecosistema Java utilizzate nella mia parte di progetto, ciascuna corredata del permalink GitHub al punto del codice in cui è impiegata.

- **Stream API** – `allMatch` con method reference (`Parameter::isAlive`).
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/model/GameEngineImpl.java#L90>
- **Stream API** – `filter`, `findFirst`, `orElseGet` per la selezione della carta non ancora utilizzata.
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/model/GameEngineImpl.java#L55>
- **Immutable collections** – `List.of()` per la lista non modificabile dei parametri.
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/model/GameEngineImpl.java#L22>
- **Static Factory Method** – `createStandard()` e `createShort()` con tipo di ritorno l’interfaccia `GameConfig`.
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/model/GameConfigImpl.java#L35>
- **Objects.requireNonNull** – fail-fast sui parametri del costruttore di `GameEngineImpl`.
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/model/GameEngineImpl.java#L40>
- **JavaFX Animation API** – `FadeTransition` per l’ingresso fluido dell’overlay `ReportImpl`.
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/view/ReportImpl.java#L78>
- **Ereditarietà da componente JavaFX** – `extends VBox` per integrare `ReportImpl` nella scena.
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/view/ReportImpl.java#L22>
- **Enum GameState** come tipo di ritorno della `State Machine`.
<https://github.com/Federicolocaa/00P25-Aurea/blob/main/src/main/java/it/unibo/aurea/model/GameEngineImpl.java#L76>

Codice di terze parti Nessuna porzione di codice della mia parte di progetto è stata tratta da fonti esterne. L’unica libreria utilizzata, oltre a Java 21 e JUnit 5, è JavaFX per il componente `ReportImpl`.

Capitolo 4

Commenti finali

In quest'ultimo capitolo si tirano le somme del lavoro svolto e si delineano eventuali sviluppi futuri.

4.1 Autovalutazione e lavori futuri

4.1.1 Pietro Dapporto

Il mio ruolo all'interno del gruppo ha riguardato soprattutto la struttura di base del gioco e la modellazione delle carte. Per questo motivo, in alcune parti del mio contributo non è stato necessario utilizzare costrutti Java particolarmente complessi: l'obiettivo principale era costruire una struttura chiara, coerente e facilmente estendibile.

La parte più creativa del lavoro, di cui sono particolarmente soddisfatto, è stata la creazione delle carte di gioco. Dal punto di vista implementativo, questa attività non ha richiesto tecnicismi avanzati, ma ha comportato un lavoro importante di progettazione del contenuto: definire le situazioni proposte al giocatore, le possibili risposte e i pesi associati agli effetti sui parametri del gioco. Ritengo inoltre di aver adottato soluzioni strutturali discrete, soprattutto per quanto riguarda la separazione tra i dati delle carte in file di configurazione esterni e la logica principale dell'applicazione.

Considero positivo il fatto di aver progressivamente migliorato l'architettura del sistema, passando da soluzioni inizialmente più semplici ma meno scalabili a una struttura più modulare e mantenibile.

Per quanto riguarda i punti deboli, il file YAML contenente le carte è diventato piuttosto esteso. Con l'aggiunta di altre carte la gestione di un unico file può diventare meno comoda. Ho valutato la possibilità di suddividerlo in più file, ma ho ritenuto che, almeno in questa fase del progetto, mantenere un unico file fosse più semplice da gestire e riducesse il rischio di frammentare eccessivamente le informazioni.

Un altro limite riguarda il livello di astrazione introdotto. In alcuni punti la struttura del codice è stata pensata per supportare possibili estensioni future, come modelli diversi di carte o di effetti, che però non sono state effettivamente sviluppate nel progetto finale. Lo stesso vale per l'utilizzo dei DTO, che separano bene la rappresentazione esterna dei dati dal model, ma introducono anche un passaggio intermedio aggiuntivo. Nel complesso, quindi, ritengo che la struttura realizzata sia abbastanza flessibile e adatta a evoluzioni future, anche se alcune di queste possibilità non sono state sfruttate pienamente nella versione attuale del progetto.

4.1.2 Mario Conventi

[Autovalutazione individuale da redigere.]

4.1.3 Federico Locatelli

[Autovalutazione individuale da redigere.]

4.1.4 Nikita Piraino

Contributi principali Ho progettato e implementato il sottosistema model del gioco, responsabile della logica di stato e del ciclo temporale: `GameConfigImpl` con il pattern Static Factory (inclusa la configurazione ridotta `createShort()` per i test), `GameClockImpl` con la gestione turno/semestre/fine partita, e `GameEngineImpl` come orchestratore centrale con la State Machine a tre stati.

Come contributo aggiuntivo al layer view ho sviluppato `ReportImpl`, il componente JavaFX che gestisce la visualizzazione dei riepiloghi inter-semesterali e della schermata di fine partita, progettato per essere riutilizzato senza creare istanze multiple.

Aspetti tecnici rilevanti L'uso dello Stream API in `areAllParametersAlive()` e in `getCurrentCard()` rappresenta una scelta consapevole di leggibilità e correttezza rispetto all'approccio imperativo. La lista `parameters` dichiarata con `List.of()` garantisce l'immutabilità strutturale del gioco dopo la costruzione dell'istanza. La separazione fra `getParameters()` (accesso diretto) e `getCopyOfParameters()` (copia difensiva) consente al controller di ottenere il riferimento necessario per le modifiche, lasciando aperta la possibilità di esporre il modello in sola lettura in futuro.

Aspetti migliorabili e lavori futuri Il metodo `start()` è attualmente privo di logica (il commento nel sorgente indica che la selezione della carta è stata spostata in `getCurrentCard()`). In una versione più matura, `start()` potrebbe inizializzare lo stato del mazzo o validare le precondizioni prima dell'inizio della partita.

La gestione dell'accesso in scrittura ai parametri da parte di attori diversi (controller, view) è ancora aperta: `getParameters()` espone la lista originale, il che consentirebbe alla view di modificare direttamente i parametri senza passare dal controller. Una soluzione futura potrebbe essere esporre solo `getCopyOfParameters()` all'esterno e introdurre un metodo `applyEffect()` nel `GameEngine` per incanalare tutte le modifiche attraverso il modello.

4.2 Difficoltà incontrate e commenti per i docenti

[Sezione opzionale: eventuali difficoltà incontrate dal gruppo nel corso o nello svolgimento del progetto, e commenti utili al miglioramento del corso. Il contenuto della sezione non impatterà il voto finale.]

Appendice A

Guida utente

[Da redigere: istruzioni d'uso essenziali per consentire ai docenti di avviare ed esplorare le funzionalità primarie dell'applicazione, eventualmente corredate di screenshot. In particolare: come avviare il JAR, come iniziare una partita, come effettuare una scelta su una carta, come interpretare la barra dei parametri, come consultare il riepilogo di fine semestre, come riconoscere la fine partita.]

Appendice B

Esercitazioni di laboratorio

In questo capitolo ciascuno studente elenca gli esercizi di laboratorio svolti, indicando i permalink dei post sul forum dove è avvenuta la consegna.

pietro.dapporto@studio.unibo.it

- lab09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287289>
- lab11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287289>

nikita.piraino@studio.unibo.it

[Permalink delle consegne di laboratorio di Nikita Piraino.]

- Laboratorio NN: <https://virtuale.unibo.it/mod/forum/discuss.php?d=...#p...>

mario.conventi@studio.unibo.it

[Permalink delle consegne di laboratorio di Mario Conventi.]

federico.locatelli3@studio.unibo.it

[Permalink delle consegne di laboratorio di Federico Locatelli.]